

Lecture 4: Linear Time Selection/Median

Michael Dinitz

September 10, 2026

601.433/633 Introduction to Algorithms

Announcements

Office hours:

- ▶ I will have office hours every Thursday 11–12 (started today).
- ▶ Prof. Sorrell will have office hours every Tuesday (4:30–5:30) (starting next week).
- ▶ Shruthi will hold recitations every Friday 12–1 in Malone 222 (starting tomorrow).
 - ▶ Small room — will change if attendance high.
- ▶ Normal CA office hours M-Th 6pm–9pm (starting next week). Still figuring out rooms.

HW1 out now! Due next Thursday by 11:59pm.

Intro and Problem Definition

Last few lectures: sorting

- ▶ Insertion sort, quicksort: $O(n^2)$
- ▶ Mergesort, heapsort: $O(n \log n)$

Today: two related problems

- ▶ Median: Given array $\mathbf{A}$ of length n , find the median: $\lceil n/2 \rceil$ 'th smallest element.
- ▶ Selection: Given array $\mathbf{A}$ of length n and $k \in [n] = \{1, 2, \dots, n\}$, find k 'th smallest element.

Intro and Problem Definition

Last few lectures: sorting

- ▶ Insertion sort, quicksort: $O(n^2)$
- ▶ Mergesort, heapsort: $O(n \log n)$

Today: two related problems

- ▶ Median: Given array $\mathbf{A}$ of length n , find the median: $\lceil n/2 \rceil$ 'nd smallest element.
- ▶ Selection: Given array $\mathbf{A}$ of length n and $k \in [n] = \{1, 2, \dots, n\}$, find k 'th smallest element.

Can solve both in $O(n \log n)$ time via sorting. Faster?

Warmup

$k = 1$:

Warmup

$k = 1$: Scan through array, keeping track of smallest. **$O(n)$** time.

Warmup

$k = 1$: Scan through array, keeping track of smallest. **$O(n)$** time.

$k = n$:

Warmup

$k = 1$: Scan through array, keeping track of smallest. **$O(n)$** time.

$k = n$: Scan through array, keeping track of largest. **$O(n)$** time.

Warmup

$k = 1$: Scan through array, keeping track of smallest. **$O(n)$** time.

$k = n$: Scan through array, keeping track of largest. **$O(n)$** time.

$k = O(1)$ or **$k = n - O(1)$** :

Warmup

$k = 1$: Scan through array, keeping track of smallest. **$O(n)$** time.

$k = n$: Scan through array, keeping track of largest. **$O(n)$** time.

$k = O(1)$ or $k = n - O(1)$: keep track of **k** smallest or **$n - k$** largest. **$O(n)$** time.

Warmup

$k = 1$: Scan through array, keeping track of smallest. $O(n)$ time.

$k = n$: Scan through array, keeping track of largest. $O(n)$ time.

$k = O(1)$ or $k = n - O(1)$: keep track of k smallest or $n - k$ largest. $O(n)$ time.

Does this work when $k = n/2$?

Warmup

$k = 1$: Scan through array, keeping track of smallest. $O(n)$ time.

$k = n$: Scan through array, keeping track of largest. $O(n)$ time.

$k = O(1)$ or $k = n - O(1)$: keep track of k smallest or $n - k$ largest. $O(n)$ time.

Does this work when $k = n/2$?

- ▶ Need to keep track of $n/2$ smallest.

Warmup

$k = 1$: Scan through array, keeping track of smallest. $O(n)$ time.

$k = n$: Scan through array, keeping track of largest. $O(n)$ time.

$k = O(1)$ or $k = n - O(1)$: keep track of k smallest or $n - k$ largest. $O(n)$ time.

Does this work when $k = n/2$?

- ▶ Need to keep track of $n/2$ smallest.
- ▶ When scanning, for each element seen, determine if it is among the k smallest. If so, remove the previous max of the $n/2$ elements tracked.
 - ▶ Not easy to do! Foreshadow: would need to use a *heap*. $\Theta(\log n)$ worst-case time.

Warmup

$k = 1$: Scan through array, keeping track of smallest. $O(n)$ time.

$k = n$: Scan through array, keeping track of largest. $O(n)$ time.

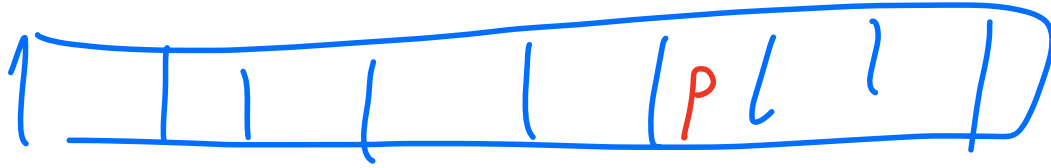
$k = O(1)$ or $k = n - O(1)$: keep track of k smallest or $n - k$ largest. $O(n)$ time.

Does this work when $k = n/2$?

- ▶ Need to keep track of $n/2$ smallest.
- ▶ When scanning, for each element seen, determine if it is among the k smallest. If so, remove the previous max of the $n/2$ elements tracked.
 - ▶ Not easy to do! Foreshadow: would need to use a *heap*. $\Theta(\log n)$ worst-case time.
- ▶ $\Theta(n \log n)$ worst-case time.

Quickselect

Main idea: Quicksort, but only recurse on side with element we're looking for.



Quickselect

Main idea: Quicksort, but only recurse on side with element we're looking for.

Quickselect(***A***, ***k***):

1. If $|A| = 1$, return the element.
2. Pick a pivot element ***p*** (somehow).
3. Compare each element of ***A*** to ***p***, creating subarrays ***L*** of elements less than ***p*** and ***G*** of elements greater than ***p***.
4. 4.1 If $|L| = k - 1$:

Quickselect

Main idea: Quicksort, but only recurse on side with element we're looking for.

Quickselect(***A***, ***k***):

1. If $|A| = 1$, return the element.
2. Pick a pivot element ***p*** (somehow).
3. Compare each element of ***A*** to ***p***, creating subarrays ***L*** of elements less than ***p*** and ***G*** of elements greater than ***p***.
4. 4.1 If $|L| = k - 1$: return ***p***.

Quickselect

Main idea: Quicksort, but only recurse on side with element we're looking for.

Quickselect(***A***, ***k***):

1. If $|A| = 1$, return the element.
2. Pick a pivot element ***p*** (somehow).
3. Compare each element of ***A*** to ***p***, creating subarrays ***L*** of elements less than ***p*** and ***G*** of elements greater than ***p***.
4.
 - 4.1 If $|L| = k - 1$: return ***p***.
 - 4.2 if $|L| > k - 1$:

Quickselect

Main idea: Quicksort, but only recurse on side with element we're looking for.

Quickselect(***A***, ***k***):

1. If $|A| = 1$, return the element.
2. Pick a pivot element ***p*** (somehow).
3. Compare each element of ***A*** to ***p***, creating subarrays ***L*** of elements less than ***p*** and ***G*** of elements greater than ***p***.
4.
 - 4.1 If $|L| = k - 1$: return ***p***.
 - 4.2 if $|L| > k - 1$: return Quickselect(***L***, ***k***).

Quickselect

Main idea: Quicksort, but only recurse on side with element we're looking for.

Quickselect(***A***, ***k***):

1. If $|A| = 1$, return the element.
2. Pick a pivot element ***p*** (somehow).
3. Compare each element of ***A*** to ***p***, creating subarrays ***L*** of elements less than ***p*** and ***G*** of elements greater than ***p***.
4.
 - 4.1 If $|L| = k - 1$: return ***p***.
 - 4.2 if $|L| > k - 1$: return Quickselect(***L***, ***k***).
 - 4.3 If $|L| < k - 1$:

Quickselect

Main idea: Quicksort, but only recurse on side with element we're looking for.

Quickselect(**A**, **k**):

1. If $|\mathbf{A}| = 1$, return the element.
2. Pick a pivot element $\mathbf{p}$ (somehow).
3. Compare each element of $\mathbf{A}$ to $\mathbf{p}$, creating subarrays $\mathbf{L}$ of elements less than $\mathbf{p}$ and $\mathbf{G}$ of elements greater than $\mathbf{p}$.
4.
 - 4.1 If $|\mathbf{L}| = k - 1$: return $\mathbf{p}$.
 - 4.2 if $|\mathbf{L}| > k - 1$: return Quickselect($\mathbf{L}$, k).
 - 4.3 If $|\mathbf{L}| < k - 1$: return Quickselect($\mathbf{G}$, $k - |\mathbf{L}| - 1$).

$$k - (|\mathbf{L}| + 1)$$

Quickselect: Correctness

Sketch here: good exercise to do at home!

Quickselect: Correctness

Sketch here: good exercise to do at home!

Prove by induction (“loop invariant”) that on any call to $\text{Quickselect}(\mathbf{X}, \mathbf{a})$, the element we’re looking for is $\mathbf{a}$ ’th smallest of $\mathbf{X}$.

- ▶ Base case: first call to $\text{Quickselect}(\mathbf{A}, \mathbf{k})$. Correct by definition.
- ▶ Inductive case: suppose the element we’re looking for is the $\mathbf{b}$ ’th smallest element of $\mathbf{Y}$ when we call $\text{Quickselect}(\mathbf{Y}, \mathbf{b})$.
 - ▶ If we return element: correct
 - ▶ If we recurse on $\mathbf{L}$: correct
 - ▶ If we recurse on $\mathbf{G}$: correct

Quickselect: running time

Main idea: depends on pivot ***p***!

Quickselect: running time

Main idea: depends on pivot ***p***!

- ▶ Suppose ***p*** is always the median, i.e., ***n*/2** smallest element.

Quickselect: running time

Main idea: depends on pivot p !

- ▶ Suppose p is always the median, i.e., $n/2$ smallest element.
 - ▶ $O(\log n)$ recursion levels, but total time linear!

Quickselect: running time

Main idea: depends on pivot p !

- ▶ Suppose p is always the median, i.e., $n/2$ smallest element.
 - ▶ $O(\log n)$ recursion levels, but total time linear!
 - ▶ $T(n) = T(n/2) + cn$

Quickselect: running time

Main idea: depends on pivot ***p***!

- ▶ Suppose ***p*** is always the median, i.e., ***n/2*** smallest element.
 - ▶ **$O(\log n)$** recursion levels, but total time linear!
 - ▶ **$T(n) = T(n/2) + cn$**
 - ▶ **$T(n) = \Theta(n)$** .

Quickselect: running time

Main idea: depends on pivot p !

- ▶ Suppose p is always the median, i.e., $n/2$ smallest element.
 - ▶ $O(\log n)$ recursion levels, but total time linear!
 - ▶ $T(n) = T(n/2) + cn$
 - ▶ $T(n) = \Theta(n)$.
- ▶ Suppose p is always smallest or largest.

Quickselect: running time

Main idea: depends on pivot p !

- ▶ Suppose p is always the median, i.e., $n/2$ smallest element.
 - ▶ $O(\log n)$ recursion levels, but total time linear!
 - ▶ $T(n) = T(n/2) + cn$
 - ▶ $T(n) = \Theta(n)$.
- ▶ Suppose p is always smallest or largest.
 - ▶ $T(n) = T(n-1) + cn$.

Quickselect: running time

Main idea: depends on pivot p !

- ▶ Suppose p is always the median, i.e., $n/2$ smallest element.
 - ▶ $O(\log n)$ recursion levels, but total time linear!
 - ▶ $T(n) = T(n/2) + cn$
 - ▶ $T(n) = \Theta(n)$.
- ▶ Suppose p is always smallest or largest.
 - ▶ $T(n) = T(n-1) + cn$.
 - ▶ $T(n) = \Theta(n^2)$

Quickselect: running time

Main idea: depends on pivot p !

- ▶ Suppose p is always the median, i.e., $n/2$ smallest element.
 - ▶ $O(\log n)$ recursion levels, but total time linear!
 - ▶ $T(n) = T(n/2) + cn$
 - ▶ $T(n) = \Theta(n)$.
- ▶ Suppose p is always smallest or largest.
 - ▶ $T(n) = T(n-1) + cn$.
 - ▶ $T(n) = \Theta(n^2)$

For any “simple” rule for picking pivot p , there is a bad array: whatever makes p always largest or smallest.

Quickselect: running time

Main idea: depends on pivot p !

- ▶ Suppose p is always the median, i.e., $n/2$ smallest element.
 - ▶ $O(\log n)$ recursion levels, but total time linear!
 - ▶ $T(n) = T(n/2) + cn$
 - ▶ $T(n) = \Theta(n)$.
- ▶ Suppose p is always smallest or largest.
 - ▶ $T(n) = T(n-1) + cn$.
 - ▶ $T(n) = \Theta(n^2)$

For any “simple” rule for picking pivot p , there is a bad array: whatever makes p always largest or smallest.

- ▶ $p = A[0]$: A already sorted (p always smallest) or A in reverse order (p always largest).

Quickselect: running time

Main idea: depends on pivot p !

- ▶ Suppose p is always the median, i.e., $n/2$ smallest element.
 - ▶ $O(\log n)$ recursion levels, but total time linear!
 - ▶ $T(n) = T(n/2) + cn$
 - ▶ $T(n) = \Theta(n)$.
- ▶ Suppose p is always smallest or largest.
 - ▶ $T(n) = T(n-1) + cn$.
 - ▶ $T(n) = \Theta(n^2)$

For any “simple” rule for picking pivot p , there is a bad array: whatever makes p always largest or smallest.

- ▶ $p = A[0]$: A already sorted (p always smallest) or A in reverse order (p always largest).
- ▶ $p = A[n-1]$: same!

Quickselect: running time

Main idea: depends on pivot p !

- ▶ Suppose p is always the median, i.e., $n/2$ smallest element.
 - ▶ $O(\log n)$ recursion levels, but total time linear!
 - ▶ $T(n) = T(n/2) + cn$
 - ▶ $T(n) = \Theta(n)$.
- ▶ Suppose p is always smallest or largest.
 - ▶ $T(n) = T(n-1) + cn$.
 - ▶ $T(n) = \Theta(n^2)$

For any “simple” rule for picking pivot p , there is a bad array: whatever makes p always largest or smallest.

- ▶ $p = A[0]$: A already sorted (p always smallest) or A in reverse order (p always largest).
- ▶ $p = A[n-1]$: same!
- ▶ $p = A[n/2]$: fun example, work it out at home!

Linear-time Version

Intuition:

- ▶ Simple pivot rule (first element, last element, etc.) bad because might not split array well.
- ▶ Can't find pivot that's exactly in the middle: median problem!
- ▶ Can we find a pivot that's “close” to the middle?

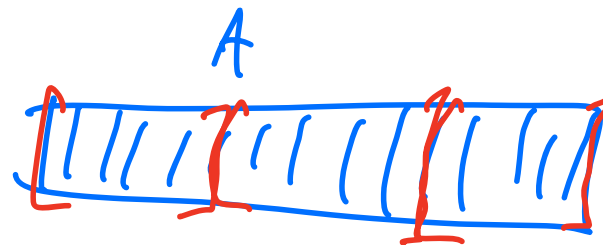
Linear-time Version

Intuition:

- ▶ Simple pivot rule (first element, last element, etc.) bad because might not split array well.
- ▶ Can't find pivot that's exactly in the middle: median problem!
- ▶ Can we find a pivot that's "close" to the middle?

Median-of-medians:

- ▶ Split **A** into $n/5$ groups of **5** elements each.
- ▶ Compute median of each group.
- ▶ Let **p** be the median of the $n/5$ medians



Linear-time Version

Intuition:

- ▶ Simple pivot rule (first element, last element, etc.) bad because might not split array well.
- ▶ Can't find pivot that's exactly in the middle: median problem!
- ▶ Can we find a pivot that's "close" to the middle?

Median-of-medians:

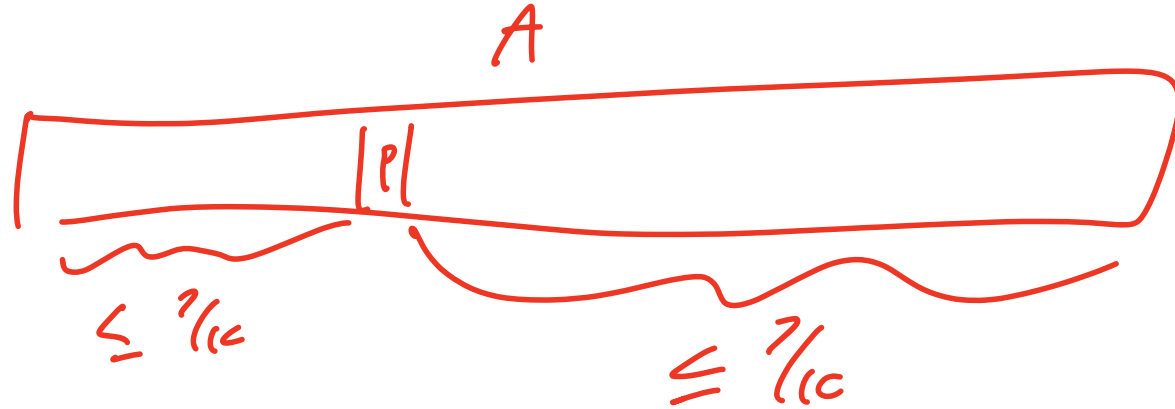
- ▶ Split **A** into **$n/5$** groups of **5** elements each.
- ▶ Compute median of each group.
- ▶ Let **p** be the median of the **$n/5$** medians

Claim: **p** is a good pivot, and can find **p** efficiently.

Median-of-Medians is good pivot

Theorem

$|L|$ and $|G|$ are both at most $7n/10$ when p is the median of medians.



Median-of-Medians is good pivot

Theorem

$|L|$ and $|G|$ are both at most $7n/10$ when p is the median of medians.

Let B be a group (of 5 elements), m median of B :

–	–	m	–	–
---	---	-----	---	---

Median-of-Medians is good pivot

Theorem

$|L|$ and $|G|$ are both at most $7n/10$ when p is the median of medians.

Let B be a group (of 5 elements), m median of B :

-	-	m	-	-
---	---	-----	---	---

- ▶ If $m < p$: at least three elements of B (m and two smaller) are in L

Median-of-Medians is good pivot

Theorem

$|L|$ and $|G|$ are both at most $7n/10$ when p is the median of medians.

Let B be a group (of 5 elements), m median of B :

–	–	m	–	–
---	---	-----	---	---

- ▶ If $m < p$: at least three elements of B (m and two smaller) are in L
- ▶ If $m > p$: at least three elements of B (m and two larger) are in G

Median-of-Medians is good pivot

Theorem

$|L|$ and $|G|$ are both at most $7n/10$ when p is the median of medians.

Let B be a group (of 5 elements), m median of B :

–	–	m	–	–

- ▶ If $m < p$: at least three elements of B (m and two smaller) are in L
- ▶ If $m > p$: at least three elements of B (m and two larger) are in G

By definition of p , exactly $n/10$ groups have $m < p$ and $n/10$ have $m > p$

Median-of-Medians is good pivot

Theorem

$|L|$ and $|G|$ are both at most $7n/10$ when p is the median of medians.

Let B be a group (of 5 elements), m median of B :

-	-	m	-	-
---	---	-----	---	---

- ▶ If $m < p$: at least three elements of B (m and two smaller) are in L
- ▶ If $m > p$: at least three elements of B (m and two larger) are in G

By definition of p , exactly $n/10$ groups have $m < p$ and $n/10$ have $m > p$

$$|L| \geq \frac{n}{10} \cdot 3 = \frac{3n}{10} \implies |G| \leq \frac{7n}{10}$$

Median-of-Medians is good pivot

Theorem

$|L|$ and $|G|$ are both at most $7n/10$ when p is the median of medians.

Let B be a group (of 5 elements), m median of B :

-	-	m	-	-
---	---	-----	---	---

- ▶ If $m < p$: at least ²three elements of B (m and two smaller) are in L
- ▶ If $m > p$: at least ²three elements of B (m and two larger) are in G

By definition of p , exactly $n/10$ groups have $m < p$ and $n/10$ have $m > p$

$$\overset{n/6}{|L|} \geq \frac{n}{10} \cdot 3 = \frac{3n}{10} \implies |G| \leq \frac{7n}{10}$$

$\frac{n}{6} \cdot 2 = \frac{n}{3}$ $\leq \frac{2n}{3}$

$$|G| \geq \frac{n}{10} \cdot 3 = \frac{3n}{10} \implies |L| \leq \frac{7n}{10} \leq \frac{2n}{3}$$

Finding Median of Medians

$n/3$

Have $n/5$ elements (median of each group). Want to find their median.

What problem is this?

Finding Median of Medians

Have $n/5$ elements (median of each group). Want to find their median.

What problem is this? Median / Selection!

Finding Median of Medians

Have $n/5$ elements (median of each group). Want to find their median.

What problem is this? Median / Selection!

Recursion! Use the same algorithm on array of medians.

BFPRT

Algorithm due to Blum-Floyd-Pratt-Rivest-Tarjan.

BFPRT

Algorithm due to Blum-Floyd-Pratt-Rivest-Tarjan.

$\text{BFPRT}(\mathbf{A}, k)$

BFPRT

Algorithm due to Blum-Floyd-Pratt-Rivest-Tarjan.

BFPRT($\mathbf{A}, k$)

1. If $|\mathbf{A}| \leq 5$, sort $\mathbf{A}$ and return the k th element.
2. Group $\mathbf{A}$ into $n/5$ groups of 5, and let $\mathbf{A}'$ be an array of size $n/5$ containing the median of each group.
3. Let $\mathbf{p} = \text{BFPRT}(\mathbf{A}', n/10)$, i.e., recursively find the median $\mathbf{p}$ of $\mathbf{A}'$ (the median-of-the-medians).

BFPRT

Algorithm due to Blum-Floyd-Pratt-Rivest-Tarjan.

BFPRT($\mathbf{A}, k$)

1. If $|\mathbf{A}| \leq 5$, sort $\mathbf{A}$ and return the k th element.
2. Group $\mathbf{A}$ into $n/5$ groups of 5, and let $\mathbf{A}'$ be an array of size $n/5$ containing the median of each group. $O(n)$ $O(n)$
3. Let $\mathbf{p} = \text{BFPRT}(\mathbf{A}', n/10)$, i.e., recursively find the median $\mathbf{p}$ of $\mathbf{A}'$ (the median-of-the-medians). $T(n/3)$ $T(n/5)$
4. Split $\mathbf{A}$ using $\mathbf{p}$ as a pivot into $\mathbf{L}$ and $\mathbf{G}$. $O(n)$ $O(n)$
5. Recurse on the appropriate piece:
 - 5.1 if $|\mathbf{L}| = k - 1$ then return $\mathbf{p}$.
 - 5.2 if $|\mathbf{L}| > k - 1$ then return $\text{BFPRT}(\mathbf{L}, k)$. $T(n/10)$ $T(\frac{2n}{3})$
 - 5.3 if $|\mathbf{L}| < k - 1$ then return $\text{BFPRT}(\mathbf{G}, k - |\mathbf{L}| - 1)$.

BFPRT Analysis

Let $T(n)$ be the (worst-case) running time on an array A of size n .

BFPRT Analysis

Let $T(n)$ be the (worst-case) running time on an array A of size n .

- ▶ Step 1:
- ▶ Step 2:
- ▶ Step 3:
- ▶ Step 4:

BFPRT Analysis

Let $T(n)$ be the (worst-case) running time on an array A of size n .

- ▶ Step 1: $O(n)$ time
- ▶ Step 2: $T(n/5)$ time $T(n/5)$
- ▶ Step 3: $O(n)$ time
- ▶ Step 4: $T(7n/10)$ time $T(7n/10)$

BFPRT Analysis

Let $T(n)$ be the (worst-case) running time on an array A of size n .

- ▶ Step 1: $O(n)$ time
- ▶ Step 2: $T(n/5)$ time
- ▶ Step 3: $O(n)$ time
- ▶ Step 4: $T(7n/10)$ time

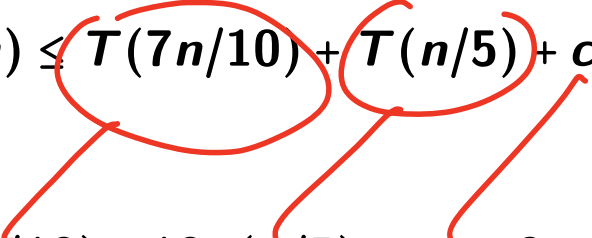
$$T(n) \leq T(7n/10) + T(n/5) + cn$$

$$T(n) \leq T(7n/10) + T(n/5) + cn = \Theta(n \log n)$$

BFPRT Analysis

Let $T(n)$ be the (worst-case) running time on an array A of size n .

- ▶ Step 1: $O(n)$ time
- ▶ Step 2: $T(n/5)$ time
- ▶ Step 3: $O(n)$ time
- ▶ Step 4: $T(7n/10)$ time

$$T(n) \leq T(7n/10) + T(n/5) + cn$$


Guess $T(n) \leq 10cn$:

$$T(n) \leq 10c(7n/10) + 10c(n/5) + cn = 9cn + cn = 10cn$$

$$7cn + 2cn + cn$$

$O(n \log n)$ Quicksort

Can use this to get $O(n \log n)$ -time Quicksort!

$O(n \log n)$ Quicksort

Can use this to get $O(n \log n)$ -time Quicksort!
Use BFPRT($A, n/2$) to choose median as pivot.

$O(n \log n)$ Quicksort

Can use this to get $O(n \log n)$ -time Quicksort!
Use BFPRT($\mathbf{A}, n/2$) to choose median as pivot.

Let $T(n)$ be time on input of size n .

- ▶ BFPRT to find pivot takes $O(n)$ time
- ▶ Splitting around pivot takes $O(n)$ time
- ▶ Each recursive call takes $T(n/2)$ time

$O(n \log n)$ Quicksort

Can use this to get $O(n \log n)$ -time Quicksort!
Use BFPRT($A, n/2$) to choose median as pivot.

Let $T(n)$ be time on input of size n .

- ▶ BFPRT to find pivot takes $O(n)$ time
- ▶ Splitting around pivot takes $O(n)$ time
- ▶ Each recursive call takes $T(n/2)$ time

$$T(n) = 2T(n/2) + cn \implies T(n) = \Theta(n \log n)$$